

# Assignment 06

## Multimodal Search System for E-Commerce

Information System Analysis and Design

**Báo cáo phân tích, thiết kế và prototype**

Sinh viên: Chưa cung cấp

MSSV: Chưa cung cấp

Lớp: Chưa cung cấp

Ngày thực hiện: 01/10/2026

Phiên bản: Local deliverable — Assignment 06

Sản phẩm: ba sơ đồ UML native trong Visual Paradigm, Python prototype ba tầng, dataset và ảnh synthetic, demo text/voice/image/fusion, kiểm thử, thực nghiệm và hướng dẫn tái lập.

Voice dùng speech-to-text mô phỏng. Image search dùng descriptor thủ công từ pixels; không sử dụng mô hình deep learning. Báo cáo đọc metrics/demo trực tiếp từ artifacts của chương trình.

# 1. Introduction và Problem Description

Khách hàng thương mại điện tử không chỉ nhập tên sản phẩm: họ có thể gõ một mô tả, nói một yêu cầu hoặc đưa ảnh tham khảo. Nếu mỗi modality tạo một hệ thống retrieval riêng không có contract thống nhất, việc bảo trì, đánh giá và kết hợp kết quả sẽ khó hơn. Assignment 06 yêu cầu thiết kế một prototype minh họa cách đưa nhiều input về chung một pipeline.

Nguyên tắc trung tâm của đề là **Different Inputs** → **Common Query Representation** → **Retrieval** → **Ranking** → **Results**. Chương trình này tách Presentation, Application / Intelligence và Data; UML mô tả đúng các trách nhiệm và dependencies. Mục tiêu là chứng minh thiết kế có thể thực thi và kiểm chứng, không xây hệ thống bán hàng production.

## Bài toán và các ví dụ

| Nhu cầu               | Input                     | Xử lý                                              |
|-----------------------|---------------------------|----------------------------------------------------|
| Tìm giày đen          | black shoes               | Normalize, keyword matching, rank                  |
| Tìm bằng lời nói      | find black running shoes  | Transcript mô phỏng → query voice → text retrieval |
| Tìm theo ảnh          | black_shoe_query.png      | Pixel descriptor → cosine → rank                   |
| Kết hợp mô tả và ảnh  | black shoes + query image | Score fusion text/image                            |
| Tìm đơn hàng của mình | O001 + C001               | Exact lookup trong customer scope                  |

## Phạm vi và giả định

Baseline gồm repository, text, simulated voice, image similarity và ranking. Extension gồm text+image fusion, category/price filters, product details và order lookup có customer context. Hệ thống chạy offline bằng CLI, dữ liệu JSON và ảnh synthetic. Voice không có microphone/ASR thật; image encoder không học từ dữ liệu.

PDF trang 15 nêu demo ít nhất hai mode, nhưng Task 6 trang 19 nêu ba query text/voice/image; bản này thực hiện đủ ba. Use case Search Order/View Order bắt buộc dù code order là extension, vì vậy prototype có lookup đơn giản. Ví dụ còn ghi Assignment 05 được chuẩn hóa thành Assignment 06 theo bìa đề. Thông tin sinh viên chưa cung cấp không được suy ra từ tài khoản máy.

## 2. Requirements Analysis

Actor là **Customer**. SpeechService, ImageService, repositories và index là thành phần nội bộ. Input gồm text, transcript voice mô phỏng và file ảnh; output gồm ranked product metadata/scores/processing hoặc order detail trong customer scope.

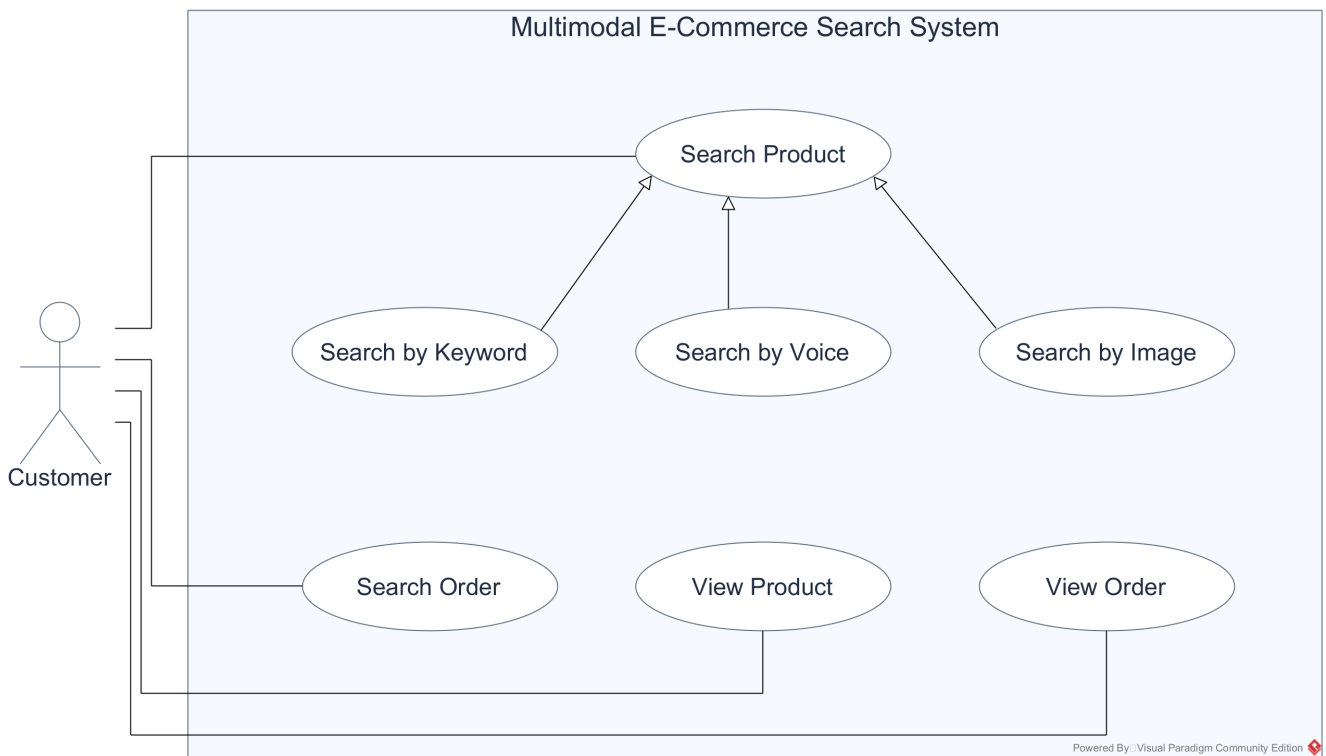
| ID    | Yêu cầu chức năng            | Nghiệm thu                                  |
|-------|------------------------------|---------------------------------------------|
| FR-01 | Keyword/text search          | Normalize, matching và kết quả có score     |
| FR-02 | Simulated voice search       | SpeechService → QueryService → retrieval    |
| FR-03 | Image similarity search      | Encode pixels cùng encoder/index, cosine    |
| FR-04 | Common query contract        | Schema thống nhất, validate modality/config |
| FR-05 | Ranking riêng và score       | Sort giảm dần, tie-break ID, top-k          |
| FR-06 | View Product                 | Hiển thị product details qua service        |
| FR-07 | Search/View Order            | Exact id lookup, customer scope, not-found  |
| FR-08 | ≥10 products + images        | Unique ids, fields/paths/index hợp lệ       |
| FR-09 | Robust input/no-match        | Lỗi rõ, không traceback cho lỗi nhập        |
| FR-10 | Text + image fusion          | Weights hợp lệ, score components công khai  |
| FR-11 | Price/category filtering     | under 100 = price <100, filters truy vết    |
| FR-12 | Reproducible demo/evaluation | Logs/JSON/metrics từ chương trình thật      |

### Yêu cầu phi chức năng

Maintainability: Presentation không đọc repository/database; retrieval và ranking tách biệt. Reproducibility: dataset/encoder/version và tie-break cố định. Robustness: query rỗng, ảnh hỏng, dimension sai, zero vector và invalid config được xử lý. Usability: --help, default demo, README và nhãn simulation rõ. Portability: paths theo project root, không phụ thuộc shortcut VP của máy tác giả. Privacy: order chỉ trả theo context đã cung cấp; context này chưa có authentication.

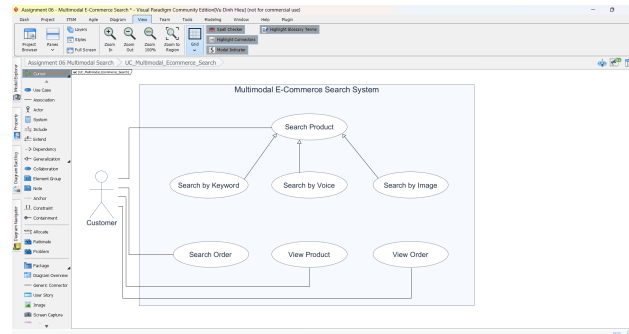
Mục tiêu hiệu năng đề xuất là ≤1 giây/query sau khi load trên dataset nhỏ. Phép đo thực tế ở trang 9; mục tiêu này không phải yêu cầu cứng trong PDF và không chứng minh tải production. Traceability chi tiết FR → use case → service → method → test/demo có trong docs/traceability.md.

### 3. Use Case Model



Hình 1. Use Case Diagram native: Customer và bảy mục tiêu bắt buộc.

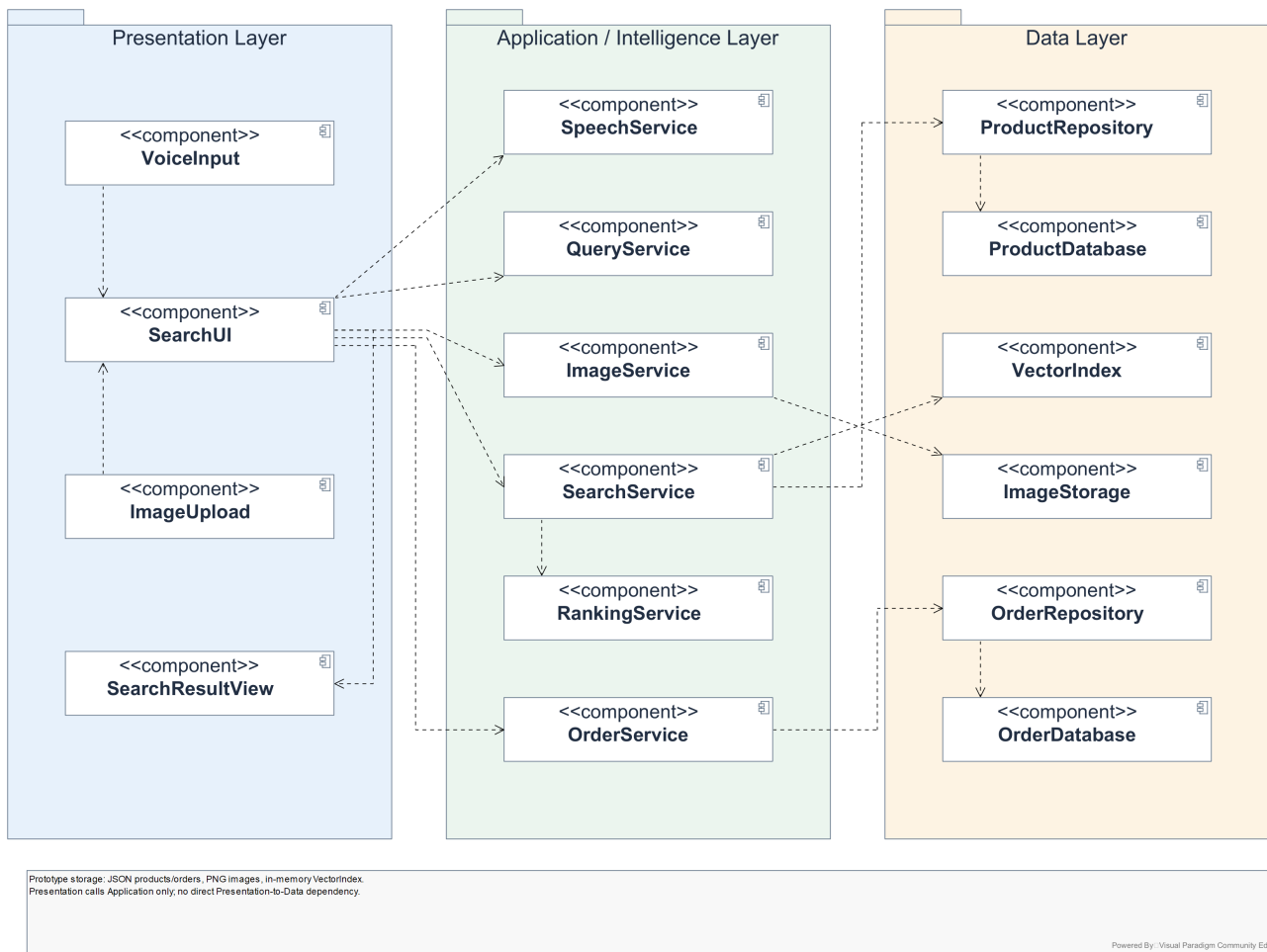
Search by Keyword, Search by Voice và Search by Image là các specialization của Search Product; generalization hướng về use case tổng quát. Customer có association đến mục tiêu tìm/xem products và orders. Không dùng include khiến một search bắt buộc chạy cả ba modality. View Product/View Order là mục tiêu xem detail sau lựa chọn của Customer.



Hình 2. Screenshot thực tế của Use Case Diagram trong Visual Paradigm.

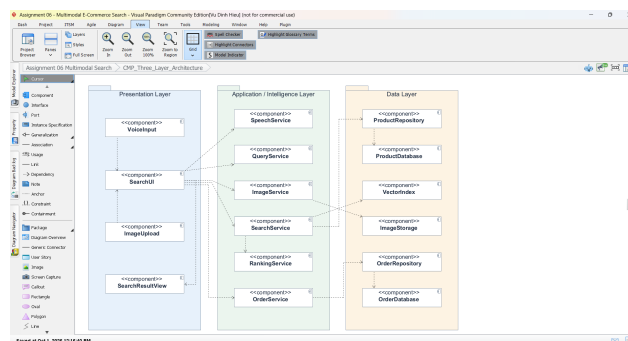
Precondition Search Product: repository/index hợp lệ và input có ít nhất một modality. Main flow: input → common query → retrieval/filter → rank → display. Alternatives: invalid input trả lỗi rõ; no-match trả list rỗng. Search/View Order yêu cầu order id + customer context, order sai id/owner trả not-found. Mô tả đầy đủ bảy use case nằm trong docs/use\_cases.md.

## 4. Three-Layer Architecture



Hình 3. Component Diagram: ba package native và dependencies theo tầng.

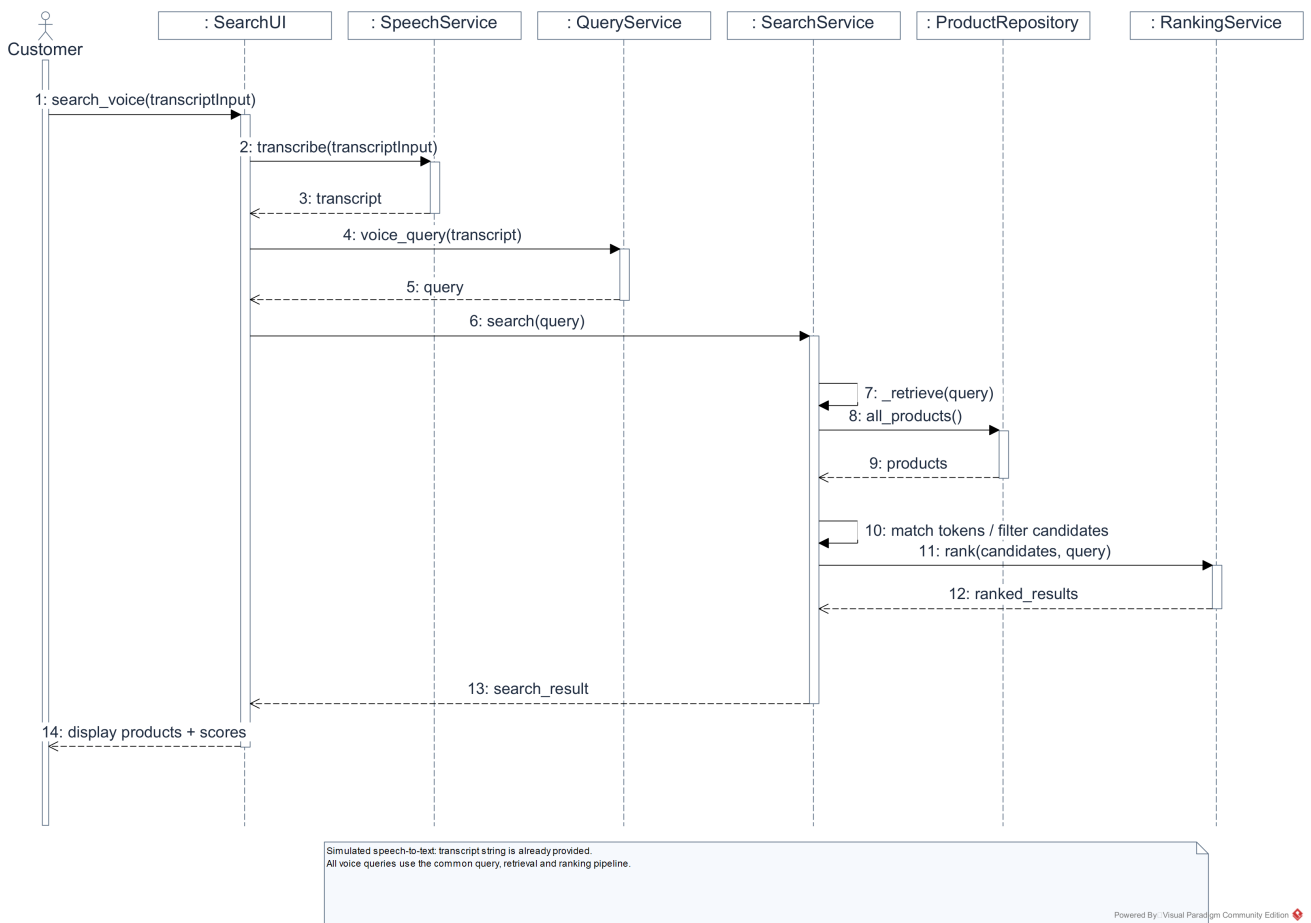
Presentation thu input và trình bày output. Application chuẩn hóa, chuyển modality, retrieval/filter và ranking. Data quản lý products/orders JSON, images và vectors. Dependencies đi từ bên dùng đến bên được dùng; không có Presentation → Data. OrderService là extension Application cho order lookup. main.py là composition root ghép dependencies, không phải UI đọc database.



Hình 4. Screenshot Component Diagram thực tế trong Visual Paradigm.

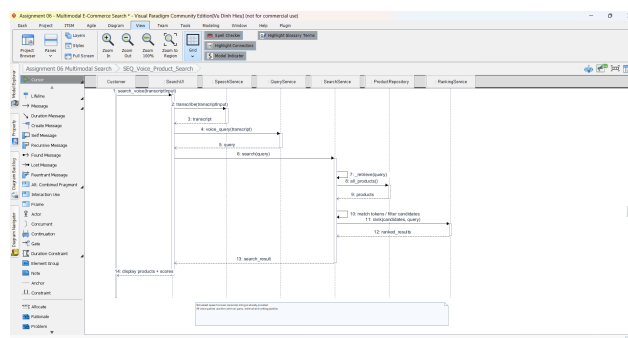
ProductDatabase/OrderDatabase là JSON storage. VectorIndex là index in-memory với metadata/vectors JSON, không phải vector database. ImageStorage là folder local. SearchService lấy candidates từ repository/index; RankingService sở hữu final scores/thứ tự/top-k. Thiết kế và các contract cụ thể có trong docs/architecture.md.

## 5. UML Sequence Diagram — Voice Search



Hình 5. Voice sequence native với bảy participants và call/return.

SearchUI nhận transcriptInput từ Customer, gọi SpeechService.transcribe rồi QueryService.voice\_query. SearchService.search dùng retrieval qua ProductRepository.all\_products, sau đó RankingService.rank. Trong code, \_retrieve giữ candidate statistics; retrieve\_candidates là public wrapper. Search result quay về UI; retrieval tạo candidates, ranking quyết định thứ tự cuối.



Hình 6. Screenshot Voice Sequence Diagram thực tế trong Visual Paradigm.

Note **Simulated STT: input is already transcribed text** mô tả implementation thực tế. Transcript được validate và giữ type=voice; chương trình không thu âm hay nhận diện âm thanh. Cùng transcript/text, filters và ranking config phải tạo cùng products/scores. Lỗi input rỗng bị từ chối trước retrieval; no-match hợp lệ trả empty results.

## 6. Python Implementation

| Package / file                 | Trách nhiệm                                               |
|--------------------------------|-----------------------------------------------------------|
| main.py                        | Composition root, argparse/CLI entry point                |
| presentation/                  | SearchUI, input adapters, ResultView                      |
| application/query_service.py   | Normalize/validate, common query và filters               |
| application/speech_service.py  | Simulated STT adapter                                     |
| application/image_service.py   | Open pixels và 88-dimensional descriptor                  |
| application/search_service.py  | Candidate retrieval/filter và orchestration               |
| application/ranking_service.py | Scores, stable sort, top-k                                |
| application/order_service.py   | Customer-scoped lookup                                    |
| data/                          | Repositories, VectorIndex, JSON/images/query fixtures     |
| scripts/, tests/               | Dataset/index preparation, demos/evaluation, verification |

### Query và result contract

Query chứa type, raw\_input, query/tokens, embedding nếu có, filters, top\_k và weights. Voice giữ type=voice dù retrieval dựa text. Embeddings phải hữu hạn và đúng dimension; top-k là integer dương. Score fusion chỉ gán weight cho modalities có input. Result envelope gồm query, processing, results; mỗi product result ghi metadata, raw\_text\_score, text\_score, image\_score, business\_score, final\_score và matched\_terms. Dữ liệu repository không bị sửa khi bổ sung scores.

Dataset có 12 products, IDs duy nhất: 12. Index rgbhist-gray-thumbnail v1.0 (88 chiều) được lưu cùng vectors trong data/embeddings.json. Hình sản phẩm/query là synthetic illustrations được sinh local, không phải ảnh chụp thương mại.

### Cài đặt và chạy tái lập

Dùng Python 3.12, tạo venv và cài requirements.txt; dev/report dependencies tách trong manifest riêng. Chạy python main.py --demo để xem các mode; dùng --mode text/voice/image/multimodal để chạy riêng. Demo chi tiết và scores đọc trực tiếp từ artifacts ở trang 10. README liệt kê lệnh cụ thể, scripts chuẩn bị index/dataset, tests và evaluation.

JSON/index validation giúp phát hiện ids trùng, paths thiếu, fields/ranges sai, metadata/dimension không phù hợp và index cũ sau thay ảnh. Script/report không sử dụng product\_id từ tên ảnh để giả lập similarity.

Kiểm thử: 78/78 tests pass, failures=0, errors=0, skipped=0 (JUnit artifact). Line coverage 100.00% trên 264 statements thuộc application và data. Không diễn giải line coverage thành branch coverage hay coverage toàn project/UI/VP.

## 7. Multimodal Search Method

### Text và voice retrieval

Text được lowercase, bỏ punctuation/stopwords và chuẩn hóa aliases có giới hạn. Matching dựa token boundaries và các query terms duy nhất trên product metadata; `raw_text_score` là số terms khớp, `text_score = matched_terms / query_terms`. Category/price được extract/validate khi hỗ trợ; under 100 có nghĩa strict <100. Voice qua SpeechService rồi dùng cùng text pipeline.

### Image representation và cosine

ImageService đọc pixels và tạo descriptor 88 chiều: RGB histogram 8 bins mỗi channel (24 chiều) kết hợp grayscale thumbnail 8×8 (64 chiều). Hai block được L2-normalize riêng, concatenate với trọng số bằng nhau, rồi L2-normalize vector cuối. Product/query dùng cùng encoder/version. Descriptor phản ánh màu/cấu trúc đơn giản, không phải learned semantic embeddings.

$$\text{cosine}(a,b) = \text{dot}(a,b) / (\text{norm}(a) \times \text{norm}(b))$$

Zero norm được xử lý bằng score 0; dimension sai hoặc NaN/Infinity bị từ chối. Sai số float được clamp; image score dùng cùng transformation nhất quán trước fusion. Similarity cao không phải xác suất product đúng.

### Fusion và ranking

$$\text{final\_score} = w_{\text{text}} \times \text{text\_score} + w_{\text{image}} \times \text{image\_score}$$

Text/voice dùng weights (1,0); image (0,1); fusion mặc định (0.5,0.5). `business_score` hiện bằng 0, không có stock-aware boost; stock chỉ là metadata hiển thị. Fusion dùng candidate union từ modalities, sau đó filter/rank; không cộng token/pixel vectors ở hai không gian khác nhau.

SearchService tạo candidates chưa sort/cắt top-k cuối. RankingService tính `final_score`, sort score giảm dần, tie-break `product_id` tăng dần và giữ top-k. Đây là sự tách biệt retrieval/ranking theo PDF trang 22. Product details và order lookup không chạy product ranking. Các query có no-match được phép trả list rỗng; invalid input là lỗi có message và exit code.

## 8. Experimental Results

Tập ground truth cố định được lưu cùng project trước evaluation. Baseline primary có 12 queries (4 text, 4 voice, 4 image); thành công khi Top-1 nằm trong relevant\_ids. Extension gồm fusion/order; challenge kiểm tra synonym/negation; robustness kiểm tra no-match, invalid input, zero vector và wrong customer. Không gộp criteria khác nhau thành retrieval accuracy.

**Success rate = successful queries / total queries**

| Mode  | Queries | Successful | Success rate |
|-------|---------|------------|--------------|
| text  | 4       | 4          | 100.00%      |
| voice | 4       | 4          | 100.00%      |
| image | 4       | 4          | 100.00%      |
| Tổng  | 12      | 12         | 100.00%      |

Nhóm riêng ngoài baseline: extension: 2/2 (100.00%); challenge: 0/2 (0.00%); robustness: 5/5 (100.00%). Không gộp robustness error-handling vào retrieval accuracy.

Thời gian processing local: mean: 1.289 ms; p95: 6.615 ms; max: 6.615 ms. Phạm vi đo là pipeline query sau khi khởi tạo services/index; không bao gồm cold startup và không đại diện tải production.

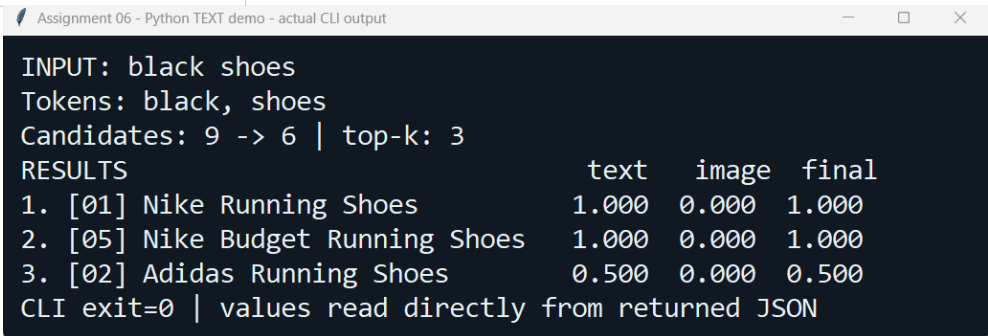
| ID / mode   | Input                             | Top-1 | Đúng? |
|-------------|-----------------------------------|-------|-------|
| T01 / text  | black shoes                       | 1     | Có    |
| T02 / text  | blue shoes                        | 4     | Có    |
| T03 / text  | black leather bag                 | 3     | Có    |
| T04 / text  | find Nike shoes under 100 dollars | 5     | Có    |
| V01 / voice | find running shoes                | 1     | Có    |
| V02 / voice | show me a white shoe              | 2     | Có    |
| V03 / voice | black leather bag                 | 3     | Có    |
| V04 / voice | green backpack                    | 12    | Có    |
| I01 / image | p001.png                          | 1     | Có    |
| I02 / image | black_shoe_query.png              | 5     | Có    |
| I03 / image | blue_shoe_query.png               | 4     | Có    |
| I04 / image | brown_bag_query.png               | 8     | Có    |

Số liệu đọc trực tiếp từ metrics.json/results.json và đối chiếu theo group. I01 dùng ảnh product giống hệt để sanity check; I02–I04 dùng biến thể synthetic nhẹ, chưa phải ảnh chụp độc lập. CSV/JSON đầy đủ nằm trong artifacts/evaluation/. Tập nhỏ này không đại diện khách hàng thật; challenge failures phân tích ở trang 11.

## 9. Demonstration

Screenshots chụp Python result viewer đọc JSON trả về từ lệnh CLI chạy mới. Log/JSON lưu input, processing, products/scores; voice ghi simulated STT. Top-2 đọc từ demo\_results.json.

| Mode / input      | Top products và final score                                             |
|-------------------|-------------------------------------------------------------------------|
| text: black shoes | 1 Nike Running Shoes (1.000000); 5 Nike Budget Running Shoes (1.000000) |

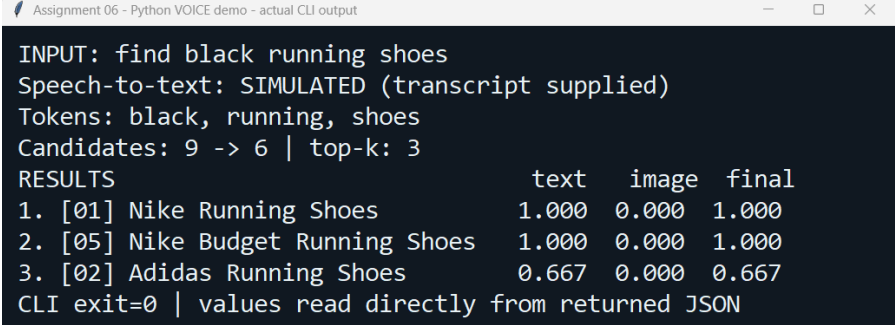
  


```
Assignment 06 - Python TEXT demo - actual CLI output

INPUT: black shoes
Tokens: black, shoes
Candidates: 9 -> 6 | top-k: 3
RESULTS
1. [01] Nike Running Shoes      text  image  final
1.000 0.000 1.000
2. [05] Nike Budget Running Shoes 1.000 0.000 1.000
3. [02] Adidas Running Shoes     0.500 0.000 0.500
CLI exit=0 | values read directly from returned JSON
```

Hình 7. Python result viewer: text input, processing, products và scores từ CLI JSON.

| Mode / input                    | Top products và final score                                             |
|---------------------------------|-------------------------------------------------------------------------|
| voice: find black running shoes | 1 Nike Running Shoes (1.000000); 5 Nike Budget Running Shoes (1.000000) |

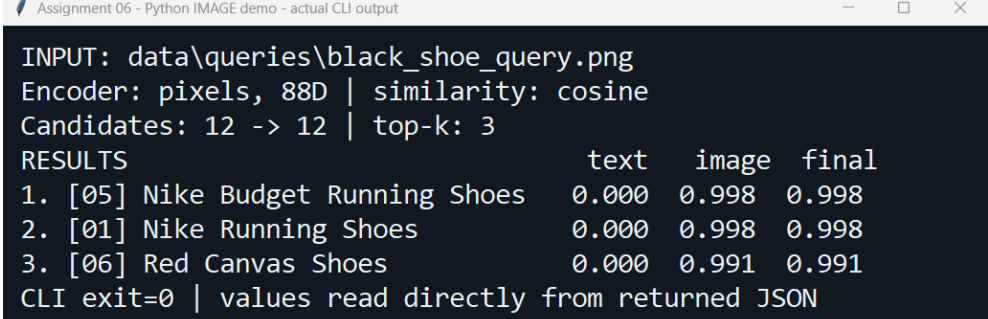
  


```
Assignment 06 - Python VOICE demo - actual CLI output

INPUT: find black running shoes
Speech-to-text: SIMULATED (transcript supplied)
Tokens: black, running, shoes
Candidates: 9 -> 6 | top-k: 3
RESULTS
1. [01] Nike Running Shoes      text  image  final
1.000 0.000 1.000
2. [05] Nike Budget Running Shoes 1.000 0.000 1.000
3. [02] Adidas Running Shoes     0.667 0.000 0.667
CLI exit=0 | values read directly from returned JSON
```

Hình 8. Python result viewer: voice transcript qua SpeechService, simulated STT.

| Mode / input                                | Top products và final score                                             |
|---------------------------------------------|-------------------------------------------------------------------------|
| image:<br>data/queries/black_shoe_query.png | 5 Nike Budget Running Shoes (0.998089); 1 Nike Running Shoes (0.998042) |

```
Assignment 06 - Python IMAGE demo - actual CLI output

INPUT: data\queries\black_shoe_query.png
Encoder: pixels, 88D | similarity: cosine
Candidates: 12 -> 12 | top-k: 3
RESULTS
1. [05] Nike Budget Running Shoes  text  image  final
0.000 0.998 0.998
2. [01] Nike Running Shoes       0.000 0.998 0.998
3. [06] Red Canvas Shoes          0.000 0.991 0.991
CLI exit=0 | values read directly from returned JSON
```

Hình 9. Python result viewer: image pixel encoding, cosine và products/scores từ CLI JSON.

Fusion/order demos và full products/scores nằm trong artifacts/demo/. Chạy lại theo README.

# 10. Discussion

## Kết quả chưa đúng quan sát được

Case C01 (text, group=challenge): input="sneakers"; ground truth=[1, 2, 4, 5, 6, 7]; Top-1=None; Top-k=[]. Unimplemented semantic synonym

Case C02 (text, group=challenge): input="shoes not black"; ground truth=[2, 4, 6, 7]; Top-1=1; Top-k=[1, 5, 2, 4, 6]. Negation unsupported by lexical OR matching

Keyword matching chỉ hiểu vocabulary/aliases đã khai báo; từ đồng nghĩa hay câu phức tạp ngoài phạm vi có thể cho no-match hoặc rank sai. Image descriptor chịu ảnh hưởng màu/nền/bố cục; sản phẩm khác ngữ nghĩa có thể gần nhau. Challenge cases giúp bộc lộ giới hạn thay vì sửa ground truth cho kết quả đẹp. Cần kiểm tra score components và ảnh/query thực tế trước kết luận nguyên nhân.

## Giới hạn thực nghiệm

Dataset và hình synthetic nhỏ, queries có thể gần nguồn ảnh sản phẩm. Success rate chỉ phản ánh criterion trên tập này, không chứng minh generalization, robustness trên ảnh chụp độc lập hay chất lượng semantic search. Độ trễ sau load không bao gồm startup và không mô phỏng concurrency. Cosine và fusion scores là heuristic, không phải probability confidence.

## Simulation, privacy và production

Voice là simulated STT; không có microphone/ASR và không đo word error rate. Image đọc pixels bằng descriptor thủ công 88 chiều; không sử dụng CLIP/CNN/semantic embeddings. Index là in-memory/JSON, không phải vector database. Customer\_id là context CLI mô phỏng; scope ngăn trả nhầm order theo context, nhưng chưa có login/authentication và người dùng có thể tự nhập context khác. Orders/products không chứa dữ liệu khách hàng thật.

Không có payment, checkout, tồn kho đồng bộ hay deployment. Không gửi private artifacts ra dịch vụ ngoài. Tên/MSSV/lớp chưa cung cấp vẫn cần điền vào cover trước nộp. Chi tiết decisions, limitations và hướng cải thiện có trong docs/decisions\_and\_limitations.md.

# 11. Conclusion và hướng phát triển

Prototype kết nối phân tích yêu cầu, UML và implementation theo pipeline thống nhất cho text, voice transcript và image. Common query giúp SearchService retrieval/filter và RankingService rank được dùng lại giữa modalities. Ba tầng làm rõ UI interaction, application logic và data storage; order extension đi qua Application service để giữ cùng nguyên tắc dependency.

Sản phẩm bàn giao gồm report PDF, project Visual Paradigm native, Python source, dataset, sample images, README và demonstration/evaluation artifacts. Việc bàn giao local không đồng nghĩa đã nộp LMS. Evaluation và demos là số liệu chương trình chạy thực tế, có thể tái lập từ README và kiểm tra từng case từ results JSON/CSV.

## Phát triển tiếp theo

- Thêm ảnh query độc lập với nền/góc/ánh sáng khác nhau; đo Recall@k/precision và phân tích errors trên dataset lớn hơn.
- Thay SpeechService bằng ASR thật, đo word error rate và downstream retrieval, giữ adapter transcript offline để regression.
- So sánh learned image/text embeddings với baseline hiện tại; công bố model/version và rebuild index bằng cùng encoder.
- Đo p50/p95 latency/memory sau load và dưới concurrency; chỉ cần nhắc vector database khi corpus đủ lớn.
- Thêm authenticated customer identity trước khi xử lý orders thật; kiểm thử authorization, logging và retention.

## Tài liệu và nguồn

1. inf\_sys\_analysis\_design\_assignment\_06\_design\_code.pdf (23 trang): task trang 17–19, evaluation/report trang 20–21, submission trang 21, rubric/principles trang 22.
2. docs/requirements.md, docs/use\_cases.md và docs/architecture.md: phân tích, use-case flows và contract thiết kế.
3. docs/traceability.md: liên kết requirements → UML/components → code → tests/demos → report.
4. artifacts/evaluation/metrics.json, results.json, results.csv và ground truth: nguồn số liệu thực nghiệm.
5. artifacts/demo/demo\_results.json và demo logs; artifacts/diagrams/ và artifacts/screenshots/: bằng chứng chạy và UML.

Các thư viện/code Python được ghi trong source và manifests. Report sinh local bằng scripts/build\_report.py, dùng Unicode font embedded; không chuyển file private qua dịch vụ PDF bên ngoài.